

AI-NATIVE DECISION FEED FOR TEAMS

TikTok for Work

Approve it, and it's already done.

Every Decision Card arrives with evidence. Every approval executes itself.

Daytona · one sandbox per decision

Neo4j · the decision graph

THE PROBLEM

An approval costs a small team twice

3

tools opened before you can say yes.

"Will this break anything? Did we decide this before? Whose work does it collide with?" lives in GitHub, Slack and last month's notes, not on the request.

2 days

lost after you say yes.

Approval is where the work starts. Someone still has to write the change, run the tests and open the PR. Between "approved" and "started", nothing moves.

Teams of 3–10 people running several businesses and repositories feel both, every day.

THE PRODUCT

You only talk to your own AI

- No channels, no DMs, no inbox. You tell your AI what you need.
- It routes the request across the organization and hands it to the right person as a **Decision Card**.
- Cards arrive in a vertical feed, one at a time. **Swipe right to approve, left to decline.**
- Outcomes sync to GitHub, filed under the business they are about.

The 3-second value: open the feed, and the decision you need to make is already there.

APPROVAL · HOTEL SAKURA · HIGH

Show tax-inclusive prices on the booking site

Requested by Toru · owner

× decline

✎ revise

✓ approve

A card that carries evidence, and an approval that ships

APPROVAL · HOTEL SAKURA · HIGH

Show tax-inclusive prices on the booking site

Requested by Toru · owner

✓ 14 tests passed · 2 files changed (+9 -3)
Verified in a Daytona sandbox in 41 s
Last similar decision: **approved by you on Aug 30**
Yui has an open card touching the same repository

× decline

✎ revise

✓ approve

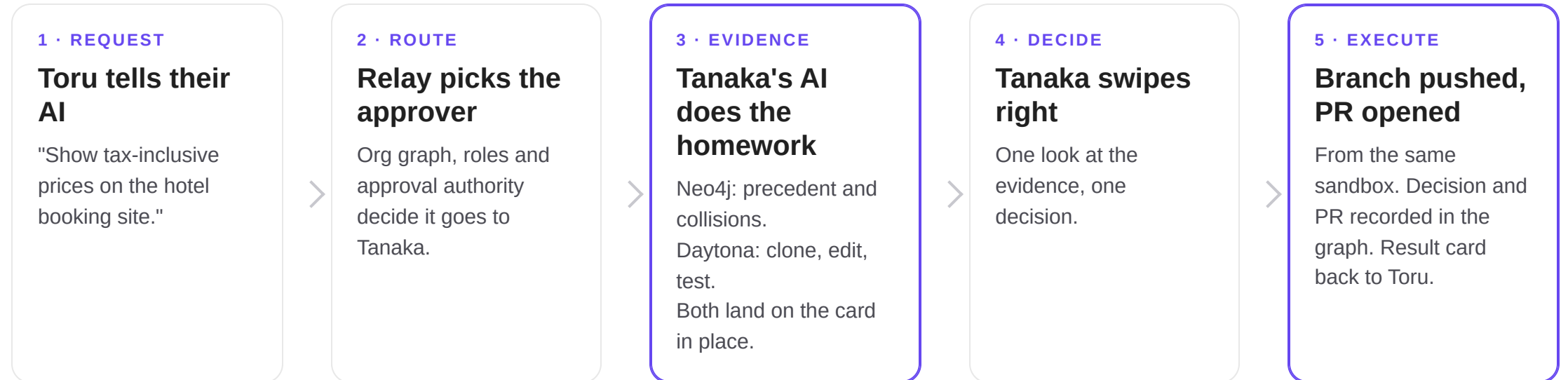
Before the recipient opens the card, their AI has already:

- **Tried it.** A fresh Daytona sandbox cloned the repo, applied the change and ran the tests.
- **Remembered.** Neo4j answered what was decided before, by whom, and whose open work this collides with.

Swipe right, and the same sandbox pushes its branch and opens the PR.

HOW IT WORKS

The recipient's AI, made real



The agent is a separate process that joins the relay **as the recipient** over the same AG-UI protocol the feed clients use. The relay lets only a card's recipient update it, which is exactly what lets the agent enrich that person's cards and nobody else's. No change to the relay or the clients.

One sandbox per decision

STEP	IN THE SANDBOX
1	<code>daytona.create()</code> , sub-second, fully isolated
2	<code>git clone</code> , <code>git checkout -b agent/card-...</code>
3	The model proposes edits; applied with <code>fs.uploadFile</code>
4	<code>npm test</code> → pass / fail counts, diff stats
5	Approve → <code>git push</code> + PR from this sandbox. Anything else → <code>sandbox.delete()</code>

Why Daytona and not a CI job or a laptop

- Cards are created continuously and in parallel. A queue would put the evidence *after* the decision.
- Sub-second, disposable environments make "the evidence is there before you look" possible.
- The sandbox outlives the dry-run: the approval pushes the exact branch that was tested.
- A failed run is still evidence. The card says so; the approver decides with it.

Decisions become context for the next decision

```
(:Person)-[:REQUESTED]->(:Decision)
(:Decision)-[:ASSIGNED_TO]->(:Person)
(:Decision)-[:DECIDED_BY {action, at}]->(:Person)
(:Decision)-[:ABOUT]->(:Business)
(:Decision)-[:TOUCHES]->(:Repo)
(:Decision)-[:PRODUCED]->(:PR)

// Precedent: what did we decide about this business?
MATCH (d:Decision {id:$id})-[:ABOUT]->(b)<-[:ABOUT]-(p)
WHERE p.action IS NOT NULL
OPTIONAL MATCH (p)-[r:DECIDED_BY]->(who)
RETURN p.title, p.action, r.at, who.login
ORDER BY r.at DESC LIMIT 5

// Collision: who is working on this code right now?
MATCH (d {id:$id})-[:TOUCHES]->(r)<-[:TOUCHES]-(o)
    -[:ASSIGNED_TO]->(p)
WHERE o.action IS NULL
RETURN o.title, p.login
```

Why a graph

- "Who approved the last three decisions about this business, what PRs came out of them, and who is touching that code now" is a multi-hop question.
- Rows can't answer it; the graph answers it in one query, per card, in milliseconds.
- Seeded from the relay's state snapshot on join. Every decision and every PR is written back as it happens.
- The rows become one sentence on the card, in the reader's language, and stay on it as data.

Two phones, one swipe, one pull request

BEAT	ON SCREEN	WHAT IT SHOWS
1	Toru's feed. Toru types the request.	Talking only to your own AI
2	Tanaka's feed. The card lands.	Routing by role and approval authority
3	Evidence appears on the card in place. Sandbox log alongside.	Daytona dry-run and Neo4j context, before anyone looked
4	Tanaka swipes right.	One decision, with evidence
5	Toru's feed: "Approved. PR #12 is open." The PR on GitHub.	The approval executed itself
6	The graph in Neo4j Browser.	The decision is now context for the next one

Built, and next

Working today

- Decision feed, routing, real-time relay, GitHub sync, per-business filing
- Agent runner: joins as the recipient, enriches cards, executes approvals
- Daytona sandbox lifecycle: create, clone, edit, test, push, PR, delete
- Neo4j graph: schema, sync from snapshot, precedent and collision queries

Next

- Evidence badges as first-class UI in the feed
- Blast radius: which other open decisions an approval changes
- Dry-runs for non-code decisions: pricing, contracts, schedules
- Graph-informed routing: precedent decides who should decide

github.com/Torutesu/Tiktokforwork